VENTION · PORTFOLIO WALKTHROUGH

Snack 4, Line by Line

A legacy quote calculator, refactored: characterization tests and a golden master first, then 13 small commits that make it typed, pure and small, every file explained.

```
> npm run demo

Quote Q-1762525200000

  40x40 extrusion 1200mm              4 ×     $35.10      $140.40
  40x80 extrusion 2400mm black        2 ×    $136.96      $273.92
  Plate PL-200                        4 ×     $32.00      $128.00
  Fasteners (pack of 50)              3 ×      $6.75       $20.25
  Linear actuator                     2 ×    $420.00      $840.00
  Motion controller                   1 ×    $650.00      $650.00
  Assembly (8 h)                      1 ×    $280.00      $280.00

  Subtotal                                               $2,332.57
  Discount                                                -$205.26
  Shipping                                                  $43.01
  Tax                                                      $282.14
  Total                                                  $2,452.46

  Ships in 17 working days
```

4 · The Legacy Refactor Kata

Projects in `Portfolio/vention/` · Code quoted directly from the repository, with file and line numbers

Contents

How to read this book 3

Chapter 4 · The Legacy Refactor Kata 4

4.1 What makes code “legacy” 5

4.2 Characterization tests and the golden master 5

4.3 Refactoring in small, green steps 6

4.4 A pure core with side effects at the edge 6

4.5 Changing behaviour on purpose 7

4.6 Money and floating point 7

Line by line 8

The code as found 8

The plan, as commits 15

Generating the golden-master cases 16

The golden master 18

Spec tests: the rules, written down 22

Named numbers 25

Types 27

One rounding function 28

Pricing each item 29

Pricing rules 33

The result 35

Interview questions 39

Exercises 41

Appendix A · Running the projects 42

Appendix B · Glossary 43

BEFORE YOU START

How to read this book

This walkthrough covers Snack 4 on its own. It's also Chapter 4 of the combined book, *Five Snacks, Line by Line*, where every chapter has the same five parts:

1. **Concepts.** The ideas the code relies on, with diagrams, before any code.
2. **Line by line.** Every source file that matters, quoted from the repository in order, with each chunk explained. The code is sliced from the real files, so the line numbers match what you'll see in your editor.
3. **Going deeper.** Boxes inside the line-by-line parts that step back to what a production system, a full engine or a large team would do differently.
4. **Interview questions.** Questions this project invites, with model answers grounded in the code.
5. **Exercises.** Extensions to try, each with a hint.

Two reading paths work well. To prepare for a conversation about the project, read the concepts and the questions. To understand or extend the code, read the line-by-line parts with the project open next to the book.

The project

	Project	Stack	What it shows
4	**The Legacy Refactor Kata**	TypeScript · node:test	A messy 180-line quote calculator made small, typed and pure in 13 commits, with characterization tests proving exactly what changed and what didn't.

CHAPTER 4 · `vention/snack4-refactor-kata`

The Legacy Refactor Kata

A messy 180-line quote calculator made small, typed and pure in 13 commits, with characterization tests proving exactly what changed and what didn't.

TypeScript · node:test

```
> npm run demo

Quote Q-1762525200000

  40x40 extrusion 1200mm              4 ×     $35.10      $140.40
  40x80 extrusion 2400mm black        2 ×    $136.96      $273.92
  Plate PL-200                        4 ×     $32.00      $128.00
  Fasteners (pack of 50)              3 ×      $6.75       $20.25
  Linear actuator                     2 ×    $420.00      $840.00
  Motion controller                   1 ×    $650.00      $650.00
  Assembly (8 h)                      1 ×    $280.00      $280.00

  Subtotal                                              $2,332.57
  Discount                                               -$205.26
  Shipping                                                $43.01
  Tax                                                    $282.14
  Total                                                 $2,452.46

  Ships in 17 working days
```

npm run demo: a quote for a sample machine design, from the refactored code.

The one idea to take away

Never change behaviour and structure at the same time. Every refactoring step goes through the same loop:

`freeze legacy` → `characterize: 1,002 cases + 21 specs` → `test the tests` → `refactor a step` → `all green` → `commit`

Then, and only then, each bug is fixed on its own, test first:

`write the correct rule as a test` → `see it fail` → `fix` → `only explained cases change` → `commit`

CONCEPTS · 4.1

What makes code “legacy”

Michael Feathers' working definition is the useful one: **legacy code is code without tests**. Age doesn't matter; what matters is that you can't change it with confidence. That creates a loop: to change code safely you want tests, and to add tests you often have to change the code.

The quote calculator breaks the loop the cheapest way: it's already a callable function, so tests can be written **around it from the outside** before a single line changes. When code isn't callable like that (hard-wired databases, globals, UI), the first steps are small, mechanical changes to create **seams**: places where behaviour can be observed or substituted.

CONCEPTS · 4.2

Characterization tests and the golden master

A **characterization test** records what the code does now, not what it should do. A **golden master** does it in bulk: generate many inputs, run them through the frozen original, and require the new code to give identical outputs.

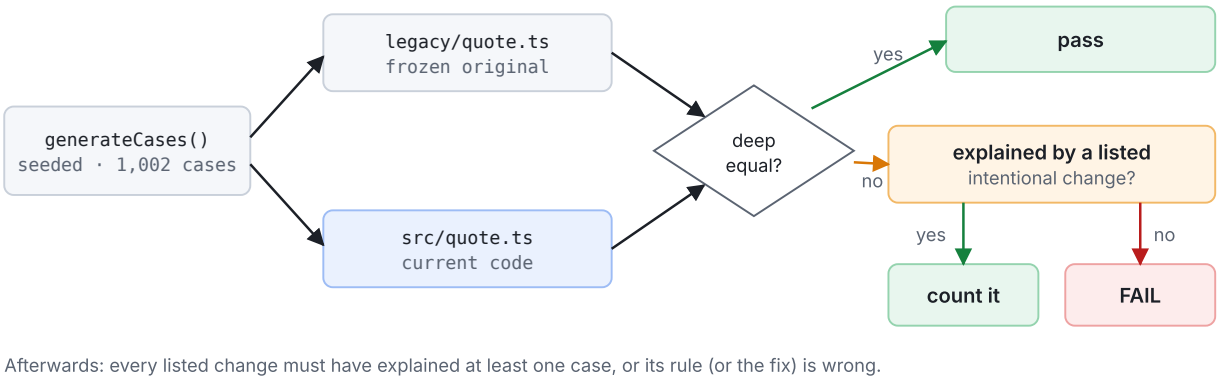


Figure 4.1. The golden master with intentional changes. Any difference must be explained, and every explanation must be used.

- **Inputs decide everything.** A seeded generator gives 1,002 varied, awkward designs that are identical on every run.
- **Check the inputs reach the code.** A coverage test asks for at least two cases through each branch worth protecting. It's how the unreachable rush lead-time check was found.
- **Test the tests.** Plant a mistake (a one-cent change to a fee) and confirm the net catches it. That's mutation testing by hand; tools like Stryker automate it.

- **Know the weakness.** A golden master locks in bugs along with everything else, and only covers what the inputs exercise. The `=== true` near-miss in commit 8 was exactly an input gap, closed in commit 9.

CONCEPTS · 4.3

Refactoring in small, green steps

Martin Fowler defines refactoring as changing the structure of code **without changing its behaviour**, in small steps. The steps used here have names in his catalog: replace magic literals with named constants, extract function, replace conditionals with table lookups, introduce types, split a calculation from its side effects.

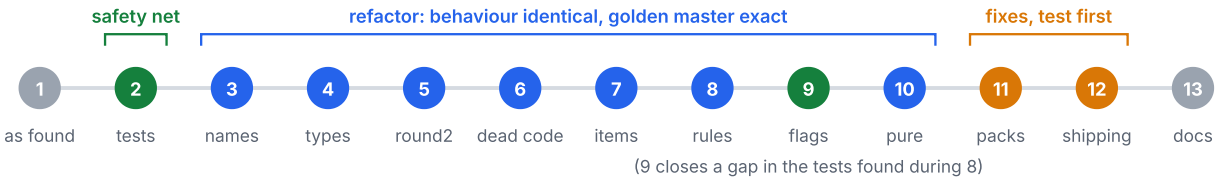


Figure 4.2. The 13 commits. Every one leaves all tests green; behaviour changes are confined to 11 and 12.

Small commits are a feature, not tidiness. Each can be reviewed in minutes, reverted alone, and found by `git bisect` if something slips through. Mixing a refactor and a fix in one commit makes the fix invisible in a sea of moved lines, and makes it impossible to revert one without the other.

CONCEPTS · 4.4

A pure core with side effects at the edge

The legacy function read the clock, wrote a global and logged. The refactored `createQuote(design, customer, now)` does none of that: same inputs, same output, always. Gary Bernhardt calls this "functional core, imperative shell": keep decisions in pure functions that are trivial to test, and push I/O to a thin outer layer.

The old entry point survives as a deprecated wrapper that adds the old side effects back. Existing callers keep working, new callers use the pure function, and the wrapper can be deleted once nobody calls it: the strangler-fig pattern at the scale of one function.

CONCEPTS · 4.5

Changing behaviour on purpose

A bug fix is a behaviour change, so it gets the opposite treatment from a refactor: write the **correct** rule as a test, watch it fail for the right reason, then fix. The golden master stays, but now with a written list of intentional changes. Each says which cases it may affect, any other difference fails, and each must explain at least one case, which proves the list is honest.

In production, the same idea scales up as **shadow mode** (GitHub's Scientist library is the well-known example): run old and new code side by side on real traffic, serve the old result, and log every mismatch until there are none you can't explain.

CONCEPTS · 4.6

Money and floating point

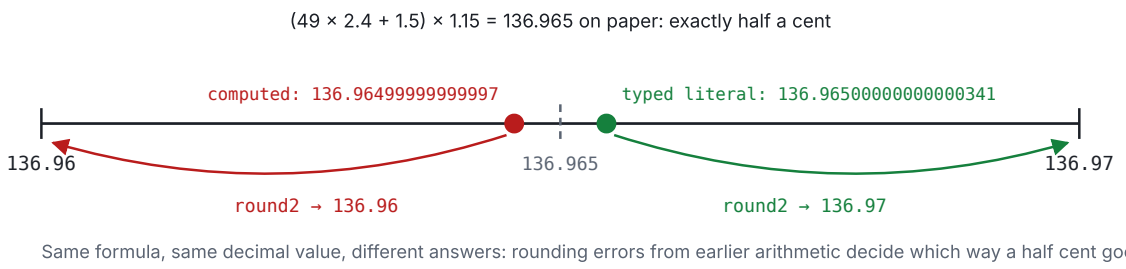


Figure 4.3. Why the demo shows 136.96. Binary floating point can't hold most decimal fractions exactly, so a computed half cent can land on either side.

JavaScript numbers are IEEE-754 doubles. They represent integers exactly up to 2^{53} but most decimal fractions only approximately, so a computed half cent can land either side of the line. The robust fixes are integer cents (or a decimal library), an explicit rounding mode (half-up, or banker's half-even), and an explicit rule for whether tax is rounded per line or on the total. All of those change invoices, which is why the refactor documented the issue instead of fixing it.

> Correction to the separate Snack 4 walkthrough: it said 136.965 itself is stored as 136.96499.... In fact the literal 136.965 is stored slightly **above** the half cent and `round2` gives 136.97. The demo's 136.96 comes from the arithmetic that produces it, as the figure shows. REFACTORING.md in the repo has the same wording.

CHAPTER 4

Line by line

11 parts, in the order data flows through the code.

PART 4.1 · `legacy/quote.ts`

The code as found

The frozen original, kept in the repo so the tests can compare against it. Reading it chunk by chunk, with the problems a reviewer would flag. None of them are fixed yet; first, the behaviour gets pinned down.

Header and globals

legacy/quote.ts · lines 5–11

```
// quote.ts - machine quote calculator
// TODO: clean this up (2019)
// DO NOT TOUCH THE ROUNDING - finance relies on it

export var lastQuote: any = null;

var PLATES: any = { "PL-100": 18.5, "PL-200": 32, "PL-300": 54.25, "BR-90": 7.8 };
```

- A five-year-old TODO and a warning not to touch the rounding: signs that people were afraid to change this file. That fear is what tests remove.
- `export var lastQuote`: **global mutable state**. Any caller can read (or overwrite) the last quote, so the function's result depends on more than its arguments, and two quotes in a row interfere.
- `any`: the type checker is switched off for everything this file touches.
- Prices hard-coded in the middle of a code file, with no name for what they are.

Signature and accumulators

legacy/quote.ts · lines 13–25

```
export function calculateQuote(design: any, customer: any) {
  var lines: any[] = [];
  var warnings: string[] = [];
  var subtotal = 0;
  var labor = 0;
  var weight = 0;
  var hours = 0;
  var hasMotion = false, hasController = false, hasActuator = false;
  var longest = 0;

  if (!design || !design.items || design.items.length == 0) {
    return { error: "Empty design" };
  }
```

Two `any` parameters, ten mutable `var`s declared up front, and three boolean flags that the loop sets and code far below reads. To follow any one number you have to keep all of them in your head. The error case returns a completely different shape from a normal quote, which callers have to know about.

Extrusions: the if-ladder

legacy/quote.ts · lines 27–49

```
for (var i = 0; i < design.items.length; i++) {
  var it = design.items[i];
  if (it.type == "extrusion") {
    var len = it.length;
    if (len < 50) {
      warnings.push("Extrusion " + it.sku + " too short (" + len + "mm), skipped");
      continue;
    }
    var perM = 0;
    var kg = 0;
    if (it.profile == "20x20") {
      perM = 12.5;
      kg = 0.5;
    } else if (it.profile == "40x40") {
      perM = 28;
      kg = 1.5;
    } else if (it.profile == "40x80") {
      perM = 49;
      kg = 2.8;
    } else {
      warnings.push("Unknown profile " + it.profile + " for " + it.sku);
      continue;
    }
```

One `for` loop and one long `if/else` chain on `it.type` handle every kind of part. Within it, another ladder picks a price and weight per profile. The numbers (12.5, 0.5, 28, 1.5, …) have no names, and `==` is used instead of `===` (loose equality, which converts types before comparing).

Extrusions: the price

legacy/quote.ts · lines 50–62

```
var price = perM * (len / 1000) + 1.5;
if (it.finish == "black") price = price * 1.15;
if (len > 3000) {
  warnings.push("Extrusion " + it.sku + " exceeds 3000mm stock; will be spliced");
  price = price + 8 + 1.5;
}
price = Math.round(price * 100) / 100;
var t = price * it.qty;
lines.push({
  sku: it.sku,
  desc: it.profile + " extrusion " + len + "mm" + (it.finish == "black" ? " black" : ""),
  qty: it.qty,
  unit: price,
```

The pricing rule is real business logic, but it's expressed only as arithmetic on unnamed numbers. Is 1.5 a cut fee? A weight? The same `1.5` is also the US shipping rate further down. Rounding happens here, mid-calculation, and again later.

Plates

legacy/quote.ts · lines 70–78

```
var p = PLATES[it.sku];
if (p == undefined) {
  warnings.push("Unknown plate " + it.sku);
  continue;
}
lines.push({ sku: it.sku, desc: "Plate " + it.sku, qty: it.qty, unit: p, total: p * it.qty });
subtotal += p * it.qty;
weight += 1.2 * it.qty;
hours += 0.25 * it.qty;
```

`PLATES[it.sku]` is an ordinary object lookup with a user-supplied key. A SKU of `"constructor"` would find the built-in `Object` constructor instead of `undefined`, and the quote would silently turn into `NaN`. Unlikely, but it's the kind of thing typed lookups prevent.

Fasteners: bug number one

legacy/quote.ts · lines 79–82

```
} else if (it.type == "fastener") {
  var packs = Math.round(it.qty / 50);
  if (packs == 0) packs = 1;
  lines.push({ sku: it.sku, desc: "Fasteners (pack of 50)", qty: packs, unit: 6.75, total:
packs * 6.75 });
```

Fasteners are sold in packs of 50, and `Math.round` rounds to the **nearest** pack. Ordering 120 bolts gives 2.4 packs, which rounds to 2: the customer is billed for 100 bolts and receives 100, twenty short. It should round **up**. The refactor keeps this behaviour (it's what the code does today) and fixes it at the end, in its own commit.

Assembly

legacy/quote.ts · lines 113–119

```
if (hasMotion && !hasController) warnings.push("No controller in design");

if (design.assembly) {
  if (hours < 2) hours = 2;
  labor = hours * 35;
  lines.push({ sku: "LAB", desc: "Assembly (" + hours + " h)", qty: 1, unit: labor, total:
labor });
}
```

Labour is billed at 35/hour with a 2-hour minimum. Note `hours` was accumulated in every branch of the loop above. This works, but only because each branch remembered to add to it.

Discounts

legacy/quote.ts · lines 121–132

```
// discounts
var disc = 0;
var tierPct = 0;
if (customer.tier == "partner") tierPct = 0.1;
if (customer.tier == "enterprise") tierPct = 0.15;
var tierDisc = subtotal * tierPct;
var promoDisc = 0;
if (customer.code == "SPRING10") promoDisc = (subtotal + labor) * 0.1;
if (promoDisc > tierDisc) disc = promoDisc;
else disc = tierDisc;
if (customer.orders > 10) disc = disc + subtotal * 0.02;
disc = Math.round(disc * 100) / 100;
```

The rules are reasonable (the larger of the tier discount or the promo code, plus a loyalty bonus), but they're spread over twelve lines of reassignments. The promo code is compared case-sensitively, so `spring10` silently does nothing. That's arguably a bug too, but nobody has asked for it to change, so it's pinned by a test and left alone.

Shipping: bug number two

legacy/quote.ts · lines 150–155

```
var ship = weight * rate;
if (ship < min) ship = min;
if ((customer.region == "CA" || customer.region == "US") && subtotal >= 2500) ship = 0;
if (customer.code == "FREESHIP") ship = 0;
if (design.rush) ship = ship * 1.5;
ship = Math.round(ship * 100) / 100;
```

Line 152 decides free shipping by comparing `subtotal` (the parts total **before** any discount) with 2,500. An enterprise customer whose order is 2,793 before and 2,374 after their 15% discount still gets free shipping, which isn't what the business intends. Again, pinned first, fixed last.

Tax

legacy/quote.ts · lines 157–168

```
// tax
var taxRate = 0;
if (customer.region == "CA") {
  if (customer.province == "ON") taxRate = 0.13;
  else if (customer.province == "QC") taxRate = 0.14975;
  else if (customer.province == "BC") taxRate = 0.12;
  else if (customer.province == "AB") taxRate = 0.05;
  else taxRate = 0.05;
} else if (customer.region == "EU") taxRate = 0.2;
var taxable = subtotal + labor - disc + ship;
var tax = Math.round(taxable * taxRate * 100) / 100;
var total = Math.round((taxable + tax) * 100) / 100;
```

Another ladder, this time for provinces. The last two branches (`AB` and the default) give the same 5%, so it's easy to misread which provinces are covered. Tax applies to the discounted goods plus shipping.

Lead time, and dead code

legacy/quote.ts · lines 170–178

```
// lead time
var lead = 5;
if (longest > 2000) lead += 3;
if (hasActuator) lead += 7;
if (design.assembly) lead += 2;
if (design.rush) {
  lead = Math.ceil(lead / 2);
  if (lead < 2) lead = 2;
}
```

Lead time starts at 5 days and only ever goes up. Halving it for a rush gives at least 3, so line 177 (`if (lead < 2) lead = 2`) can never run. Nobody noticed by reading. It was found by the coverage test described below, which asked for cases at the 2-day minimum and got none.

Side effects

legacy/quote.ts · lines 180–194

```
  var q = {
    quoteId: "Q-" + Date.now(),
    lines: lines,
    subtotal: Math.round((subtotal + labor) * 100) / 100,
    discount: disc,
    shipping: ship,
    tax: tax,
    total: total,
    leadTimeDays: lead,
    warnings: warnings,
  };
  lastQuote = q;
  console.log("quote " + q.quoteId + " total " + total);
  return q;
}
```

Three things that make the function hard to test and reuse:

- `Date.now()` in the ID: the same inputs give a different result every millisecond.
- `lastQuote = q`: the global is overwritten on every call.
- `console.log`: every quote writes to the log, whether the caller wants it or not.

GOING DEEPER

Reading code you're afraid of

Before testing, it helps to understand. Two techniques from Feathers: **scratch refactoring**, where you refactor freely on a throwaway branch just to learn the code, then delete it; and **effect sketches**, a quick diagram of which values each part of the function reads and writes. Both are how the list of problems in this part was found.

PART 4.2 · `git log`

The plan, as commits

Each step is its own commit, and every commit leaves all tests passing. The whole history ships with the project in `history/refactor-steps.bundle`.

Safety net first

1. **Add the legacy quote calculator, as found.** The starting point.
2. **Add characterization tests before touching anything.** Freeze a copy, build the golden master and spec tests, and check that the net actually catches a planted mistake.

Refactor: change structure, never behaviour

3. **Name the magic numbers:** every price, weight and rate moves into `catalog.ts`.
4. **Introduce types; let/const and a switch** instead of `any`, `var` and if-chains.
5. **Extract `round2()`** for the six copies of the rounding formula.
6. **Remove the unreachable minimum rush lead time.**
7. **Price each item type in its own function** (`items.ts`).
8. **Extract the pricing rules** (`rules.ts`).
9. **Golden master: also send non-boolean flags.** This closes a gap found in the previous step.
10. **Separate the pure calculation from its side effects:** `createQuote()` is pure, and `calculateQuote()` becomes a deprecated wrapper.

After each one: `npm test` (golden master + spec tests) and `tsc --strict`, then commit.

Fix: change behaviour, on purpose, one bug at a time

11. **Fix: fasteners are sold in whole packs, rounded up.** Test first, seen failing, then the fix.
12. **Fix: judge the free-shipping threshold after discounts.** Same pattern.

Keeping refactors and fixes in separate commits means each can be reviewed on its own and reverted on its own. A reviewer can see that commits 3–10 change no numbers at all, and that 11 and 12 each change exactly one rule.

PART 4.3 · `tests/cases.ts`

Generating the golden-master cases

Hand-written examples only check what you think of. A generator produces a thousand varied designs, including the awkward ones.

Repeatable randomness

tests/cases.ts · lines 14–27

```
/** Seeded generator, so the case list is identical on every run. */
function makeRandom(seed: number) {
  let s = seed >>> 0;
  const next = () => {
    s = (Math.imul(s, 1664525) + 1013904223) >>> 0;
    return s / 2 ** 32;
  };
  return {
    next,
    int: (min: number, max: number) => min + Math.floor(next() * (max - min + 1)),
    pick: <T>(items: readonly T[]): T => items[Math.floor(next() * items.length)],
    chance: (p: number) => next() < p,
  };
}
```

A tiny linear congruential generator with a fixed seed, so the same 1,002 cases come out on every run and on every machine. `Math.random()` would make a failure impossible to reproduce. `int`, `pick` and `chance` are the three helpers everything else uses.

Choosing values that find edges

tests/cases.ts · lines 29–46

```
const PROFILES = ["20x20", "40x40", "40x80", "40x80", "30x30"] as const;
const LENGTHS = [30, 49, 50, 120, 300, 750, 1000, 1500, 2000, 2001, 2500, 2999, 3000, 3001,
4200] as const;
const PLATES = ["PL-100", "PL-200", "PL-300", "BR-90", "PL-999"] as const;
const TIERS = ["standard", "partner", "enterprise", undefined] as const;
const REGIONS = ["CA", "CA", "US", "EU", "OTHER", "JP"] as const;
const PROVINCES = ["ON", "QC", "BC", "AB", "NS", undefined] as const;
const CODES = [undefined, undefined, "SPRING10", "spring10", "FREESHIP"] as const;

export function generateCases(count = 1000, seed = 2024): Case[] {
  const r = makeRandom(seed);
  const cases: Case[] = [
    { name: "empty design", design: { items: [] }, customer: { region: "CA" } },
    { name: "no design", design: null, customer: { region: "CA" } },
  ];

  // Callers don't always send real booleans; the legacy code used truthiness, so test that
too.
  const flag = (p: number): unknown => (r.chance(0.05) ? r.pick([1, "yes", 0, ""]) :
r.chance(p));
```

The value lists are where the thinking goes. Lengths include both sides of every boundary in the code (49/50 for the minimum, 2000/2001 for the long-piece surcharge, 2999/3000/3001 for splicing). There's an unknown profile (`30x30`), an unknown plate (`PL-999`), the lower-case promo code, an unknown region (`JP`) and province (`NS`), plus two edge cases up front: an empty design and no design at all.

`flag` is the fix from commit 9: 5% of the time, `assembly` and `rush` get a non-boolean value like `1` or `""`, because the legacy code tested truthiness and real callers might rely on that.

PART 4.4 · `tests/golden.test.ts`

The golden master

The core safety net. There's no spec, so the legacy behaviour is the spec: every case must give the same answer from the old code and the new.

Setup

tests/golden.test.ts · lines 13–28

```
import { after, before, mock, test } from "node:test";
import assert from "node:assert/strict";
import { isDeepStrictEqual } from "node:util";
import { calculateQuote as legacyQuote } from "../legacy/quote";
import { calculateQuote } from "../src/quote";
import type { Customer, Design } from "../src/types";
import { generateCases, type Case } from "./cases";

const FIXED_NOW = 1_700_000_000_000;
const cases = generateCases();

before(() => {
  mock.method(Date, "now", () => FIXED_NOW); // quote IDs use the clock
  mock.method(console, "log", () => {}); // the legacy code logs every quote
});
after(() => mock.restoreAll());
```

Both implementations are imported side by side: the frozen copy as `legacyQuote`, and the code being refactored. `mock.method` from Node's built-in test runner replaces `Date.now` with a fixed value, so the quote IDs match, and silences the legacy `console.log`. `after` puts the real ones back.

Intentional changes, written down

tests/golden.test.ts · lines 33–54

```
interface IntentionalChange {
  name: string;
  /** Could this fix change this case's quote? */
  mayAffect(c: Case, before: Any, after: Any): boolean;
}

const INTENTIONAL_CHANGES: IntentionalChange[] = [
  {
    name: "fasteners round up to whole packs",
    // Only quantities where rounding to nearest and rounding up disagree: 51-74, 101-124, ...
    mayAffect: (c) =>
      itemsOf(c).some((i) => i.type === "fastener" && Math.max(1, Math.round(i.qty / 50)) !==
Math.max(1, Math.ceil(i.qty / 50))),
  },
  {
    name: "free-shipping threshold uses the discounted order value",
    // Only CA/US orders without the free-shipping code, where free shipping applied before or
applies now.
    mayAffect: (c, before, after) => {
      const cust = c.customer as Any;
      return ["CA", "US"].includes(cust.region) && cust.code !== "FREESHIP" && (before.shipping
=== 0 || after.shipping === 0);
    },
  },
];
```

Before the bug fixes, this list was empty and the test required every case to be identical. After them, each fix is listed with a rule for **which cases it could possibly affect**:

- Rounding up only differs from rounding to nearest for some quantities: 51–74, 101–124, and so on.
- The threshold fix can only matter for CA/US orders without the free-shipping code, where shipping was free before or is free now.

This turns “the numbers changed, but I think that's expected” into something checkable.

The test

tests/golden.test.ts · lines 56–68

```
test(`current code matches legacy on all ${cases.length} cases, apart from intentional
changes`, () => {
  const changedBy = new Map(INTENTIONAL_CHANGES.map((ch) => [ch.name, 0]));
  for (const c of cases) {
    const before = legacyQuote(c.design, c.customer) as Any;
    const after = calculateQuote(c.design as Design, c.customer as Customer) as Any;
    if (isDeepStrictEqual(after, before)) continue;

    const reasons = INTENTIONAL_CHANGES.filter((ch) => ch.mayAffect(c, before, after));
    assert.ok(reasons.length > 0, `${c.name} changed, and no intentional change explains it`);
    for (const r of reasons) changedBy.set(r.name, changedBy.get(r.name)! + 1);
  }
  for (const [name, count] of changedBy) assert.ok(count > 0, `"${name}" changed no cases at
all`);
});
```

For each case, compare old and new with `isDeepStrictEqual`: every field, every line and warning, every number to the last bit. If they differ, at least one intentional change must explain it, or the test fails with the case's name. Finally, each listed fix must have changed at least one case. A fix that changed nothing means its rule, or the fix itself, is wrong.

Before the fixes this passed with all 1,002 identical. After them, 226 cases differ, and every one is accounted for.

Testing the tests

tests/golden.test.ts · lines 70–76

```
test("the cases exercise every branch worth protecting", () => {
  const seen = new Map<string, number>();
  const hit = (key: string) => seen.set(key, (seen.get(key) ?? 0) + 1);

  for (const c of cases) {
    const q = legacyQuote(c.design, c.customer) as Any;
    if ("error" in q) {
```

tests/golden.test.ts · lines 96–102

```
  const expected = [
    "error", "too short", "spliced", "unknown profile", "unknown plate", "unknown type", "no
controller",
    "assembly", "black finish", "discount", "free shipping", "free-shipping threshold", "promo
code", "tax",
  ];
  for (const key of expected) assert.ok((seen.get(key) ?? 0) >= 2, `only ${seen.get(key) ?? 0}
cases hit "${key}"`);
});
```

A golden master is only as good as its cases. This second test runs the legacy code over the same cases and counts how often each interesting branch is hit (errors, splices, unknown parts, promo codes, free shipping, tax). Each must be hit at least twice.

> This is how the dead code was found: the first version of this test also asked for the 2-day minimum lead time, and no case could reach it.

The net was also checked directly: changing the cut fee from 1.50 to 1.51 makes the golden master fail at once.

GOING DEEPER

How strict is “deeply equal”?

`isDeepStrictEqual` compares primitives with `Object.is`, so `NaN` equals `NaN` but `0` and `-0` differ, and it compares object prototypes too, while ignoring key order. That's strict enough to catch a reordered floating-point sum that changes the last bit of a number, which is why the refactor kept the legacy order of additions.

PART 4.5 · `tests/spec.test.ts`

Spec tests: the rules, written down

The golden master says “nothing changed” but not what the rules are. These 21 tests are the missing documentation, each with the arithmetic in its name.

Helpers

tests/spec.test.ts · lines 14–25

```
const NOW = 1_700_000_000_000;
afterEach(() => mock.restoreAll());

type Any = Record<string, any>;
const customer = (over: Any = {}) => ({ tier: "standard", region: "US", orders: 0, ...over });
// Items are written as plain objects for readability; cast once here.
const quote = (items: Any[], design: Any = {}, cust: Any = {}) =>
  createQuote({ items, assembly: false, rush: false, ...design } as unknown as Design,
customer(cust) as Customer, NOW) as Any;

const extrusion = (profile: string, length: number, qty = 1, finish = "clear") =>
  ({ type: "extrusion", sku: "EX", profile, length, qty, finish });
const close = (a: number, b: number) => assert.ok(Math.abs(a - b) < 1e-9, `${a} ≠ ${b}`);
```

`customer()` and `quote()` fill in sensible defaults so each test only states what matters. Since commit 10 they call the pure `createQuote` with a fixed `NOW`, so no mocking is needed. Items are written as plain objects for readability, and cast once in the helper.

A rule as an example

tests/spec.test.ts · lines 29–32

```
test("extrusion: profile price per meter × length, plus a 1.50 cut fee (40x40, 1 m: 28 + 1.5 =
29.50)", () => {
  const q = quote([extrusion("40x40", 1000, 2)]);
  assert.equal(q.lines[0].unit, 29.5);
  assert.equal(q.lines[0].total, 59);
```

The name is the specification: price per meter times length, plus a 1.50 cut fee, with the arithmetic spelled out. Anyone changing prices later can read the rule here rather than reverse-engineering it.

A bug fix, test first

tests/spec.test.ts · lines 112–120

```
test("free shipping at 2,500 is judged on what the customer pays, after discounts (was before,
a bug)", () => {
  const sevenActuators = [{ type: "actuator", sku: "LA", qty: 7 }]; // 2,793 in parts, 24.5 kg
  // Standard customer: 2,793 ≥ 2,500, ships free.
  assert.equal(quote(sevenActuators).shipping, 0);
  // Enterprise: 2,793 − 418.95 (15%) = 2,374.05 < 2,500, so pays 24.5 kg × 1.50 = 36.75.
  const enterprise = quote(sevenActuators, {}, { tier: "enterprise" });
  assert.equal(enterprise.discount, 418.95);
  assert.equal(enterprise.shipping, 36.75);
});
```

Until commit 12 this test said the opposite: its name started with “BUG (fixed later)” and it asserted the wrong behaviour, free shipping, so the refactor couldn't change it by accident. To fix the bug, the test was rewritten to state the correct rule **first**, run and seen failing (`expected 36.75, actual 0`), and only then was the code changed. It covers both sides of the line.

Purity, and the old API

tests/spec.test.ts · lines 152–168

```
test("createQuote is pure: the ID comes from the `now` argument, nothing is logged or stored",
() => {
  const log = mock.method(console, "log", () => {});
  const design = { items: [{ type: "plate", sku: "PL-100", qty: 1 }] } as Design;
  const a = createQuote(design, customer(), NOW) as Any;
  assert.equal(a.quoteId, "Q-1700000000000");
  assert.deepEqual(createQuote(design, customer(), NOW), a, "same inputs, same quote");
  assert.equal(log.mock.callCount(), 0);
});

test("the deprecated calculateQuote keeps its old side effects: clock ID, global lastQuote, log
line", () => {
  mock.method(Date, "now", () => NOW);
  const log = mock.method(console, "log", () => {});
  const q = calculateQuote({ items: [{ type: "plate", sku: "PL-100", qty: 1 }] } as Design,
customer()) as Any;
  assert.equal(q.quoteId, "Q-1700000000000");
  assert.equal(lastQuote, q);
  assert.deepEqual(log.mock.calls[0].arguments, ["quote Q-1700000000000 total 48.5"]);
});
```

Two tests pin down commit 10. `createQuote` is pure: the same inputs give an equal result, and nothing is logged. `mock.method` records calls, so `callCount()` proves it. The deprecated `calculateQuote` still has every legacy side effect, including the exact log line, so existing callers can't tell the difference.

PART 4.6 · `src/catalog.ts`

Named numbers

Commit 3. Every number the business cares about gets a name, a unit, and one place to live.

Prices and weights

src/catalog.ts · lines 1–30

```
/**
 * Every price, weight, rate and threshold the quote uses, in one place.
```

```
 * Money in dollars, weight in kg, length in mm, labour in hours, lead time in days.
 */

export const PROFILES = {
  "20x20": { pricePerMeter: 12.5, kgPerMeter: 0.5 },
  "40x40": { pricePerMeter: 28, kgPerMeter: 1.5 },
  "40x80": { pricePerMeter: 49, kgPerMeter: 2.8 },
} as const;

export const EXTRUSION = {
  cutFee: 1.5,
  spliceFee: 8,
  minLengthMm: 50,
  stockLengthMm: 3000,
  blackFinishMultiplier: 1.15,
  laborHours: 0.5,
} as const;

export const PLATE_PRICES: Readonly<Record<string, number>> = {
  "PL-100": 18.5,
  "PL-200": 32,
  "PL-300": 54.25,
  "BR-90": 7.8,
};

export const PLATE = { kg: 1.2, laborHours: 0.25 } as const;

export const FASTENER_PACK = { size: 50, price: 6.75, kg: 0.4 } as const;
```

The profile ladder becomes a table: adding a 40×120 profile is now one line, not another `else if`. `as const` makes the objects read-only and keeps their exact values in the types.

Rates and thresholds

src/catalog.ts · lines 47–60

```
export const DISCOUNTS = {
  tierRates: { partner: 0.1, enterprise: 0.15 } as Readonly<Record<string, number>>,
  promoCode: "SPRING10",
  promoRate: 0.1,
  loyaltyAfterOrders: 10,
  loyaltyRate: 0.02,
} as const;

export const SHIPPING = {
  regions: {
    CA: { perKg: 1.2, min: 25 },
    US: { perKg: 1.5, min: 30 },
    EU: { perKg: 2.4, min: 45 },
  } as Readonly<Record<string, { perKg: number; min: number }>>,
```

Discount and shipping rules are configuration now. The `as Readonly<Record<...>>` on the inner tables says they're looked up by arbitrary strings (a customer's region, say), which is exactly what the next helper is for.

Safe lookups

src/catalog.ts · lines 82–85

```
/** Reads a table entry by a key from outside (user input), ignoring inherited properties like
"constructor". */
export function lookup<T>(table: Readonly<Record<string, T>>, key: unknown): T | undefined {
  return typeof key === "string" && Object.hasOwn(table, key) ? table[key] : undefined;
}
```

`Object.hasOwn` checks that the key is the table's own, not an inherited one like `constructor` or `toString`. For every real input this behaves exactly like the old `==` checks, which is what the golden master confirmed, but junk input now finds nothing instead of a function.

PART 4.7 · `src/types.ts`

Types

Commit 4. The shapes of the inputs and outputs, written down where the compiler can check them.

One interface per item type

src/types.ts · lines 9–18

```
export interface ExtrusionItem {
  type: "extrusion";
  sku: string;
  /** "20x20", "40x40" or "40x80"; anything else is skipped with a warning. */
  profile: string;
  /** mm */
  length: number;
  qty: number;
  finish?: "clear" | "black";
}
```

Comments carry the units (mm) and the rules (other profiles are skipped). `profile` is a `string`, not a union of the three valid values, on purpose: callers do send unknown profiles, and the code has to keep handling them.

A discriminated union

src/types.ts · lines 51–57

```
export type Item = ExtrusionItem | PlateItem | FastenerItem | MotorItem | ActuatorItem |
ControllerItem;

export interface Design {
  items: readonly Item[];
  assembly?: boolean;
  rush?: boolean;
}
```

`Item` is one of six interfaces, told apart by their `type` field. In a `switch (item.type)`, TypeScript narrows `item` in each `case`, so `item.length` is only allowed in the extrusion branch, and a typo like `"extrution"` is a compile error rather than a silently skipped part.

PART 4.8 · `src/money.ts`

One rounding function

Commit 5.

round2

src/money.ts · lines 1–9

```
/**
 * Rounds to the cent. This is exactly the legacy formula, so every result stays
 * identical; that's what the old "DO NOT TOUCH THE ROUNDING" comment protects.
 * (Rounding half-cents of floating-point values isn't perfect; changing it would
 * be a behaviour change for finance to sign off, not part of a refactor.)
 */
export function round2(amount: number): number {
  return Math.round(amount * 100) / 100;
}
```

Six copies of `Math.round(x * 100) / 100` become one function with exactly the same formula, so results are bit-for-bit identical.

The comment records something the refactor deliberately did **not** change. Floating-point numbers can't represent most decimals exactly. In the demo, a black 40×80 piece is (49 × 2.4 + 1.5) × 1.15, which is 136.965 on paper. In floating point the multiplications come out at 136.96499999999997, a hair under the half cent, so it rounds down to 136.96 instead of up. Moving to whole cents would fix that, but it changes invoices, and that's a decision for whoever owns billing, not a refactoring step.

PART 4.9 · `src/items.ts`

Pricing each item

Commit 7. The 90-line loop body becomes six small functions behind one dispatcher.

The shape of a result

src/items.ts · lines 7–39

```
/** What one design item adds to a quote. */
export interface PricedItem {
  line: QuoteLine;
  weightKg: number;
  laborHours: number;
  /** Notes that don't stop the item being quoted, like a splice. */
  warnings: string[];
}

/** An item that can't be quoted, and why. */
```

```
export interface SkippedItem {
  skipped: string;
}

export function priceItem(item: Item): PricedItem | SkippedItem {
  switch (item.type) {
    case "extrusion":
      return priceExtrusion(item);
    case "plate":
      return pricePlate(item);
    case "fastener":
      return priceFasteners(item);
    case "motor":
      return priceMotor(item);
    case "actuator":
      return priceActuator(item);
    case "controller":
      return priceController(item);
    default:
      // Not a known Item, but callers can still send one.
      return { skipped: "Unknown item type " + (item as { type: unknown }).type };
  }
}
```

Each pricing function returns either a `PricedItem` (its quote line, weight, labour hours and any warnings) or a `SkippedItem` with the reason. The caller checks which with `"skipped" in priced`. That replaces the legacy `continue` statements scattered through the loop.

The `default` branch covers item types TypeScript doesn't know about. They can't happen according to the types, but can happen at runtime, so the cast `(item as { type: unknown })` is explicit about that.

Extrusions, readable

`src/items.ts · lines 41–70`

```
/** Cut to length from stock: price per meter plus a cut fee; long pieces are spliced from two.
*/
function priceExtrusion(item: ExtrusionItem): PricedItem | SkippedItem {
  const len = item.length;
  if (len < EXTRUSION.minLengthMm) return { skipped: `Extrusion ${item.sku} too short
(${len}mm), skipped` };
  const profile = lookup(PROFILES, item.profile);
```

```
  if (!profile) return { skipped: `Unknown profile ${item.profile} for ${item.sku}` };

  const warnings: string[] = [];
  const black = item.finish === "black";
  let price = profile.pricePerMeter * (len / 1000) + EXTRUSION.cutFee;
  if (black) price = price * EXTRUSION.blackFinishMultiplier;
  if (len > EXTRUSION.stockLengthMm) {
    warnings.push(`Extrusion ${item.sku} exceeds 3000mm stock; will be spliced`);
    price = price + EXTRUSION.spliceFee + EXTRUSION.cutFee;
  }
  price = round2(price);

  return {
    line: {
      sku: item.sku,
      desc: `${item.profile} extrusion ${len}mm${black ? " black" : ""}`,
      qty: item.qty,
      unit: price,
      total: price * item.qty,
    },
    weightKg: profile.kgPerMeter * (len / 1000) * item.qty,
    laborHours: EXTRUSION.laborHours * item.qty,
    warnings,
  };
}
```

The same arithmetic as lines 50–62 of the legacy file, in the same order, now with names: `pricePerMeter`, `cutFee`, `blackFinishMultiplier`, `spliceFee`. Early `return`s handle the two ways an extrusion can be skipped, so the rest of the function is the normal case.

Fasteners, fixed

src/items.ts · lines 83–98

```
/** Sold in whole packs, rounded up; quantity on the line is packs, not pieces. */
function priceFasteners(item: FastenerItem): PricedItem {
  const packs = Math.max(1, Math.ceil(item.qty / FASTENER_PACK.size));
  return {
    line: {
      sku: item.sku,
      desc: "Fasteners (pack of 50)",
      qty: packs,
      unit: FASTENER_PACK.price,
      total: packs * FASTENER_PACK.price,
    },
    weightKg: FASTENER_PACK.kg * packs,
    laborHours: 0,
    warnings: [],
  };
}
```

This is the whole of fix number one: `Math.ceil` instead of `Math.round`, with `Math.max(1, …)` keeping the old “at least one pack” rule. Because the function is small and isolated, the fix is one line and obviously correct.

GOING DEEPER

switch, table or polymorphism?

- **A table** when only data differs between cases: profiles and plates are lookups in `catalog.ts`.
- **A switch over a discriminated union** when behaviour differs and the set of cases is closed: TypeScript narrows the type in each branch and can prove the switch exhaustive.
- **Polymorphism** (a class or strategy per type) when cases are many, grow often, or come from plugins. Here it would add ceremony without benefit.

PART 4.10 · `src/rules.ts`

Pricing rules

Commit 8. The business rules that apply to the whole order, each a small pure function with a doc comment.

Labour and discount

src/rules.ts · lines 5–21

```
/** Billed assembly hours (never under the minimum) and their cost. */
export function assemblyLabor(hours: number): { hours: number; cost: number } {
  const billed = hours < ASSEMBLY.minHours ? ASSEMBLY.minHours : hours;
  return { hours: billed, cost: billed * ASSEMBLY.ratePerHour };
}

/**
 * The larger of the customer's tier discount (on parts) and the promo code (on
 * parts and labour), plus a loyalty bonus on parts. Rounded to the cent.
 */
export function discount(parts: number, labor: number, customer: Customer): number {
  const tier = parts * (lookup(DISCOUNTS.tierRates, customer.tier) ?? 0);
  const promo = customer.code === DISCOUNTS.promoCode ? (parts + labor) * DISCOUNTS.promoRate :
0;
  let amount = promo > tier ? promo : tier;
  if ((customer.orders ?? 0) > DISCOUNTS.loyaltyAfterOrders) amount = amount + parts *
DISCOUNTS.loyaltyRate;
  return round2(amount);
}
```

`discount` reads almost like the sentence in its comment: the larger of tier and promo, plus loyalty, rounded. Every expression is the same as the legacy code's, in the same order, so the floating-point results match exactly.

Shipping

src/rules.ts · lines 23–37

```
/**
 * Weight × the region's rate, with a minimum charge. Free in CA/US once
 * `orderValue` reaches the threshold, or with the free-shipping code; ×1.5 for
 * rush orders. Rounded to the cent.
 */
export function shipping(weightKg: number, orderValue: number, customer: Customer, rush:
boolean): number {
  const zone = lookup(SHIPPING.regions, customer.region) ?? SHIPPING.elsewhere;
  let cost = weightKg * zone.perKg;
  if (cost < zone.min) cost = zone.min;
  const freeRegion = customer.region !== undefined &&
SHIPPING.freeShippingRegions.includes(customer.region);
  if (freeRegion && orderValue >= SHIPPING.freeShippingThreshold) cost = 0;
  if (customer.code === SHIPPING.freeShippingCode) cost = 0;
  if (rush) cost = cost * SHIPPING.rushMultiplier;
  return round2(cost);
}
```

The parameter is called `orderValue` on purpose. During the refactor, `createQuote` passed the pre-discount parts total, as the legacy code did. Fix number two changed only the argument at the call site, not this function, so the shipping rules themselves never had to change.

Lead time

src/rules.ts · lines 53–61

```
/** Working days to ship: a base, plus extra for long pieces, actuators and assembly; rush
halves it. */
export function leadTimeDays(f: LeadTimeFactors): number {
  let days: number = LEAD_TIME.baseDays;
  if (f.longestPieceMm > LEAD_TIME.longPieceMm) days += LEAD_TIME.longPieceDays;
  if (f.hasActuator) days += LEAD_TIME.actuatorDays;
  if (f.assembly) days += LEAD_TIME.assemblyDays;
  if (f.rush) days = Math.ceil(days / 2);
  return days;
}
```

Without the dead minimum, the rush rule is one line. `LeadTimeFactors` (lines 46–51) names the four things lead time depends on, instead of reaching into the design and the loop's flags.

PART 4.11 · `src/quote.ts`

The result

Commit 10, plus fix two. The entry point now reads as a summary of the process.

A pure function

src/quote.ts · lines 6–13

```
/**
 * Prices a machine design for a customer.
 *
 * Pure: no globals, no logging, and the clock is a parameter, so the same inputs
 * always produce the same quote (which is also what makes it easy to test).
 */
export function createQuote(design: Design | null | undefined, customer: Customer, now:
number): Quote | QuoteError {
  if (!design || !design.items || design.items.length === 0) return { error: "Empty design" };
```

`now` is a parameter instead of a hidden call to `Date.now()`. That one change makes the function **pure**: same inputs, same output, no side effects. Tests don't need to mock anything, and two quotes can be computed in any order, or in parallel.

1. Add up the items

src/quote.ts · lines 15–44

```
// 1. Price every item, adding up money, weight and labour as we go.
const lines: QuoteLine[] = [];
const warnings: string[] = [];
let parts = 0;
let weightKg = 0;
let laborHours = 0;
let longestPieceMm = 0;
let hasMotion = false;
let hasActuator = false;
let hasController = false;

for (const item of design.items) {
  const priced = priceItem(item);
  if ("skipped" in priced) {
    warnings.push(priced.skipped);
```

```
    continue;
  }
  lines.push(priced.line);
  warnings.push(...priced.warnings);
  parts += priced.line.total;
  weightKg += priced.weightKg;
  laborHours += priced.laborHours;

  if (item.type === "motor" || item.type === "actuator") hasMotion = true;
  if (item.type === "actuator") hasActuator = true;
  if (item.type === "controller") hasController = true;
  if (item.type === "extrusion" && item.length > longestPieceMm) longestPieceMm = item.length;
}

if (hasMotion && !hasController) warnings.push("No controller in design");
```

Each item is priced by `priceItem`; the loop only adds up money, weight and hours, and notes the flags lead time and the controller warning need. `for…of` replaces the indexed `for` loop.

2–3. Labour, discount, shipping, tax

src/quote.ts · lines 46–73

```
  // 2. Assembly labour, if asked for.
  let labor = 0;
  if (design.assembly) {
    const billed = assemblyLabor(laborHours);
    labor = billed.cost;
    lines.push({ sku: "LAB", desc: `Assembly (${billed.hours} h)`, qty: 1, unit: labor, total:
labor });
  }

  // 3. Discount, shipping, tax.
  const rush = Boolean(design.rush);
  const disc = discount(parts, labor, customer);
  // Free-shipping threshold: what the customer pays for parts and assembly, after discounts.
  const ship = shipping(weightKg, parts + labor - disc, customer, rush);
  const taxable = parts + labor - disc + ship;
  const tax = round2(taxable * taxRate(customer));

  return {
    quoteId: `Q-${now}`,
```

```
    lines,
    subtotal: round2(parts + labor),
    discount: disc,
    shipping: ship,
    tax,
    total: round2(taxable + tax),
    leadTimeDays: leadTimeDays({ longestPieceMm, hasActuator, assembly:
Boolean(design.assembly), rush }),
    warnings,
  };
}
```

Each step is a call to a named rule. Line 58 is fix number two: the free-shipping threshold is checked against what the customer pays after discounts (`parts + labor - disc`). `Boolean(design.rush)` keeps the legacy truthiness, which commit 9's extra cases now test.

The old API, still working

src/quote.ts · lines 75–90

```
// ---- Legacy API, kept so existing callers keep working ----

/** @deprecated The most recent quote from calculateQuote(). Use its return value instead. */
export let lastQuote: Quote | null = null;

/**
 * @deprecated Use createQuote(design, customer, Date.now()).
 * Same result, plus the legacy side effects: it records the quote in `lastQuote` and logs it.
 */
export function calculateQuote(design: Design | null | undefined, customer: Customer): Quote |
QuoteError {
  const quote = createQuote(design, customer, Date.now());
  if ("error" in quote) return quote;
  lastQuote = quote;
  console.log(`quote ${quote.quoteId} total ${quote.total}`);
  return quote;
}
```

`calculateQuote` keeps its exact signature and side effects (the clock, the `lastQuote` global, the log line) but is now three lines around `createQuote`. `@deprecated` makes editors strike it through, nudging callers to migrate, while nothing breaks today. Once no caller uses it, it can be deleted in a commit of its own.

The golden master still runs through this wrapper, so it covers both: the wrapper and the pure function behind it.

GOING DEEPER

Shipping a behaviour change safely

Fixes 11 and 12 change real prices. In a real business the code is the easy part: someone owns the pricing rules and signs off, customer-facing changes may need notice, and the rollout might go behind a feature flag or run in shadow mode first. The intentional-change list in the golden master doubles as the change note: it says exactly which quotes move, and why.

CHAPTER 4

Interview questions

Q1 **How do you refactor code that has no tests?**

Pin its current behaviour first with characterization tests: a golden master over many generated inputs against a frozen copy, plus readable spec tests for the important rules. Check the inputs reach every branch, and plant a mistake to prove the tests catch it. Then change structure in small commits, each green, and only afterwards change behaviour, one bug per commit, test first.

Q2 **What's a golden master, and what are its weaknesses?**

Recorded outputs of the existing code for many inputs, compared against the new code. It locks in bugs as well as features, is only as good as the inputs, gives opaque failures ("case 731 differs") and needs an explicit mechanism for intended changes. That's why it's paired with spec tests that state the rules in words.

Q3 **How did you know the generated cases were good enough?**

A second test asserts that the cases hit every branch worth protecting at least twice, and mutation by hand: a one-cent change to a fee had to fail the suite. When a near-miss showed a gap (only real booleans were ever generated), the generator was fixed before the code was trusted again.

Q4 **Why separate refactoring commits from bug-fix commits?**

Reviewability and reversibility. A reviewer can verify that commits 3–10 change no output at all, and that 11 and 12 each change exactly one rule. Either fix can be reverted alone, and `git bisect` lands on a small, single-purpose commit.

Q5 **Why keep the deprecated `calculateQuote` ?**

Existing callers depend on its side effects: the global `lastQuote`, the log line, the clock-based ID. Removing them is a breaking change. The wrapper keeps old callers working while new code uses the pure `createQuote`, and it can be deleted when the last caller migrates.

Q6 **You found the rounding issue. Why not fix it?**

Because it isn't a refactoring question. How invoices round is a business rule, owned by whoever owns billing, and changing it moves real prices. The right move is to document it precisely and raise it, not to slip it into a refactor.

Q7 What was the `=== true` near-miss?

While extracting the rules, it would have been natural to write `design.rush === true`, but the legacy code used truthiness, so `1` or `"yes"` counted as rush. The golden master didn't object because every generated case used real booleans. The fix went into the tests first (generate non-boolean flags), and only then was the code trusted.

Q8 How would you make the clock-based quote ID testable?

Inject the time: `createQuote` takes `now` as a parameter, so tests pass a fixed number. The legacy path is tested by mocking `Date.now` (and `console.log`) in the golden master, and a spec test checks the wrapper's side effects explicitly.

Q9 How would you bring this approach to a team?

Start with the safety net in its own PR, so reviewers can trust it before anything changes. Then small refactor PRs that each say “no behaviour change, golden master exact”, and separate fix PRs with the failing test first and the list of affected cases. The commit history in the repo is that process, step by step.

CHAPTER 4

Exercises

4.1 A new item type

Add a cable chain priced per meter with a minimum length. Write the spec tests first; the golden master must stay exactly as it is.

Hint. New behaviour for new inputs only: the generator doesn't produce the new type, so legacy cases can't change.

4.2 Integer cents

Convert all money to integer cents internally, add it to the intentional-change list, and report how many of the 1,002 cases change.

Hint. Write the `mayAffect` rule first. It should be narrow: cases where a computed half cent lands on the other side.

4.3 Mutation testing

Run Stryker on `src/` and add tests for any surviving mutants that matter.

Hint. Some survivors are equivalent mutants (changes that can't alter behaviour). Decide which, and say why.

4.4 Property-based tests

With fast-check, assert properties for random designs: total = subtotal − discount + shipping + tax to the cent; discount is never negative; adding an item never lowers the subtotal.

Hint. Reuse the generator's building blocks so properties see the same awkward values as the golden master.

4.5 Regions as data

Replace the region and province rules with a table of per-region strategies (rate, shipping per kg, minimum). The golden master must stay exact.

Hint. Keep the order of floating-point operations identical, or the deep-equality check will tell you.

APPENDIX A

Running the projects

Every project needs Node.js 20 or newer, and Snack 1 needs 22. From `Portfolio/vention/`:

Project	Commands
`snack1-robot-arm`	`npm ci` · `npm run dev` (http://localhost:5173) · `npm test` · `npm run check` (type-check + tests) · `npm run build:gripper` regenerates the .glb · needs Node 22 (Appendix C)
`snack2-control-panel`	`npm install` · `npm run dev` · `npm test` (IK and program tests) · `npm run build`
`snack3-conveyor-physics`	`npm install` · `npm run dev` · `npm test` · `npm run bench` (headless physics benchmark)
`snack4-refactor-kata`	`npm install` · `npm test` (spec + golden master) · `npm run typecheck` · `npm run demo`
`snack5-inventory`	`docker compose --profile postgres up --build` (or `--profile mongo`), then http://localhost:8081 · `docker compose --profile test run --rm --build api-test` runs the contract tests on both databases · without Docker: `cd api; npm install; npm test`

Snack 4's full commit history ships as `history/refactor-steps.bundle`: `git clone history/refactor-steps.bundle steps`, then `git log --stat` in the new folder.

APPENDIX B

Glossary

AABB
Axis-aligned bounding box: a box whose faces line up with the world axes, described by its center and half sizes or its min and max corners.

Accessor (glTF)
Says how to read a range of a bufferView as numbers: component type, element type (VEC3, SCALAR…), count, bounds.

ACID
Atomicity, consistency, isolation, durability: the guarantees a database transaction gives.

Bin packing
Packing items into the fewest fixed-size bins; cut lists into stock bars are an instance.

Broad phase
The cheap first pass of collision detection that finds pairs which might touch.

Buffer view (glTF)
A byte range within a buffer, usable directly as a GPU buffer.

Characterization test
A test that records what code currently does, to detect changes, rather than what it should do.

CHECK constraint
A rule on a table that the database enforces on every write, rejecting rows that break it.

Conditional update
An update whose filter contains the rule (“only if enough is available”), so check and write are one atomic step.

Connection point
A named spot on a kind of object, stored as fractions of its size, where it can be joined to another.

Contract test
One test suite run against every implementation of an interface, so they behave the same.

CORS
Cross-origin resource sharing: headers by which a server lets pages from other origins read its responses in a browser.

Counting sort
A non-comparison sort that buckets items by a small integer key in O(n).

CTE
Common table expression (WITH … AS): names a sub-query; in Postgres it can also chain data-changing statements.

Deadlock
Two transactions each waiting for a lock the other holds; avoided by locking in a consistent order.

Deadlock
A state where each party waits for something only another waiting party can provide, so none can proceed.

Denavit–Hartenberg
A standard way to describe each joint of a robot arm with four parameters, giving one transform matrix per joint.

Diverter
A device on a conveyor that steers items off the line to one side, here a hinged flap.

Document database
A database storing self-contained JSON-like documents, which can embed arrays and sub-objects (MongoDB).

Draw call
One request from the CPU to the GPU to draw geometry with a given state. Each has fixed overhead.

Emissive
Light a material gives off regardless of the scene's lights; used to make the selected housing glow.

Event delegation
One listener on a container handling events from its children, identified by attributes, so re-rendered children need no new listeners.

First-fit decreasing
A bin-packing heuristic: sort items largest first, put each in the first bin with room.

Fixed time step
Advancing a simulation in equal small steps, independent of the display's frame rate, so results don't depend on it.

Foreign key
A column that must match a key in another table, so references are always valid.

Forward kinematics
Computing the tool's position from joint angles. Always one answer.

Frustum culling
Skipping objects whose bounding sphere is outside the camera's view.

Golden master
Recorded outputs of existing code for many inputs, compared against new code.

Idempotency key
A client-chosen key that makes retrying a request safe: the server returns the stored result instead of acting twice.

Impulse
An instant change in momentum, the solver's unit of correction.

Instancing
Drawing one geometry many times in one draw call, with per-instance data such as a matrix and color.

Interpolation (α)
Drawing a moment between the last two physics states, by the fraction α of a step left over.

Inverse kinematics
Computing joint angles that put the tool at a target. Zero, several or infinitely many answers.

Island
A group of bodies connected by contacts; they sleep and wake together.

Isolation level
How much concurrent transactions can see of each other (READ COMMITTED, REPEATABLE READ, SERIALIZABLE).

Jacobian
The matrix of partial derivatives of tool position with respect to joint angles; maps joint speeds to tool speed.

jsonb
PostgreSQL's binary JSON type: stores a document in a column, with operators and indexes to query inside it.

Kanban
Pull-based production: a step makes or sends an item only when the next step signals it needs one.

Kerf
The width of material a saw blade removes with each cut.

Kinematic body
A body moved by code, with a velocity, that pushes others but is never pushed itself.

Kinematic simulation
Motion set directly (positions and velocities) rather than computed from forces.

Kustomize overlay
A directory that builds on a base set of Kubernetes manifests, adding or changing resources.

Little-endian
Byte order with the least significant byte first; required by the glTF container.

Local frame
A coordinate system attached to an object, in which its own geometry is simple; points are transformed into it.

Lost update
A race where two transactions read the same value and one's write silently overwrites the other's.

moveJ / moveL
Robot moves that interpolate joint angles (curved tool path) or the tool position (straight line).

Narrow phase
The exact collision test on candidate pairs, producing contact normal and depth.

Normal
A unit vector perpendicular to a surface; drives lighting and, in physics, the direction of contact forces.

Normalization
Storing each fact once in its own table, related by keys, instead of repeating it.

Object pool
Preallocated objects or slots reused instead of created and destroyed.

ON DELETE CASCADE
A foreign-key option that deletes the referencing rows when the referenced row is deleted.

Origin
The scheme, host and port of a page (http://localhost:5173); browsers isolate pages by origin.

PBR
Physically based rendering: materials described by base color, metalness and roughness, lit consistently in any environment.

Preflight
The OPTIONS request a browser sends before a cross-origin request that isn't simple, asking whether it's allowed.

Pure function
A function whose output depends only on its inputs and which has no side effects.

Quaternion
Four numbers representing a 3D rotation, free of gimbal lock and easy to interpolate.

Raycasting
Intersecting a ray (here from the camera through the mouse) with scene geometry, for picking.

Re-parenting
Moving an object to a new parent in the scene graph; `attach` keeps its world pose, `add` keeps its local one.

Readiness probe
A Kubernetes check that decides whether a pod receives traffic; separate from liveness, which decides restarts.

Reconciliation
React's comparison of a new render's output with the previous one, to find the DOM changes to make.

Refactoring
Changing the structure of code without changing its behaviour, in small steps.

Replica set
A group of MongoDB servers holding the same data; required for multi-document transactions.

Repository pattern
Hiding storage behind an interface of domain operations, so the rest of the code doesn't know the database.

Ring buffer
A fixed array used as a queue by wrapping an index, so nothing is shifted or allocated.

Row lock
A lock on specific rows (SELECT … FOR UPDATE) that makes other writers wait until the transaction ends.

Rule engine
A small interpreter that evaluates conditions stored as data, instead of decisions written as code.

Saga
A multi-step operation made of individually atomic steps, with compensating steps to undo earlier ones on failure.

Scene graph
A tree of objects whose transforms compose from parent to child.

Seam
A place where code's behaviour can be observed or substituted without editing it, which makes testing possible.

Semi-implicit Euler
Integration that updates velocity first, then position from the new velocity; stable and cheap.

Sequential impulses
Solving contact constraints one at a time and repeating, a form of projected Gauss–Seidel.

Shock propagation
Treating the lower body of a vertical contact as immovable on a final, bottom-up pass, so stacks settle.

Singularity
A pose where the arm loses the ability to move the tool in some direction; the Jacobian loses rank.

Slop
Overlap allowed between resting bodies before position correction acts, to prevent jitter.

Smoothstep
The easing curve $3t^2 - 2t^3$: from 0 to 1 with zero slope at both ends.

Spatial hash
A uniform grid of cells stored in a hash table, for finding nearby objects in O(n).

Spiral of death
A slow frame causing more physics steps, making the next frame slower; avoided by capping steps.

Stand-in object
A temporary value built only so existing logic (here, rule matching) can be applied to something that doesn't exist yet.

StatefulSet
A Kubernetes workload with stable pod names and a persistent volume per pod, used for databases.

Structure of arrays
Storing each property of many entities in its own array, instead of one object per entity.

TCP
Tool center point: the point on the tool that motion is planned for; here, between the fingertips.

Tearing (React)
Different components showing different versions of external data in the same commit.

Tessellation
Converting exact surfaces (from CAD) into triangles for rendering.

Upsert
Insert a row, or update it if one with the same key exists, in one statement (INSERT ... ON CONFLICT DO UPDATE).

Winding order
The order a triangle's corners are listed; counter-clockwise as seen marks the front face.